

Compiler Design and Construction

CSE-3202: Lab Report

By

ARNISHA AKHTER

ID: 118422

Semester: 3RD YEAR 2ND SEMESTER

Session: 2010-2011

Submitted to

Md. Ashraf Uddin

Lecturer



Department of Computer Science and Engineering

Jagannath University

Dhaka, Bangladesh

December, 2014

Chapter 1	LEX			
	1.1	Introduction.....		4
		1.1.1	Overview of Flex.....	4
		1.1.2	Lexical Analysis with Flex.....	5
		1.1.3	How to use Flex.....	7
	1.2	Example of Flex.....		9
		1.2.1	A lex program to recognize and count the number of identifiers in a given input.....	9
			1.2.1.1 Problem Description.....	9
			1.2.1.2 Program Code.....	10
			1.2.1.3 Result and Discussion.....	10
		1.2.2	A lex program to count the characters, words, spaces and lines in a given input file.....	11
			1.2.2.1 Problem Description.....	11
			1.2.2.2 Program Code.....	12
			1.2.2.3 Result and Discussion.....	12
		1.2.3	A lex program to count the numbers of comment lines in a given C/C++/JAVA program.....	13
			1.2.3.1 Problem Description.....	13
			1.2.3.2 Program Code.....	14
			1.2.3.3 Result and Discussion.....	14
		1.2.4	A lex program to identify an integer number.....	15
			1.2.4.1 Problem Description.....	15
			1.2.4.2 Program Code.....	15
			1.2.4.3 Result and Discussion.....	15
		1.2.5	A lex program to identify a Teletalk number.....	16
			1.2.5.1 Problem Description.....	16
			1.2.5.2 Program Code.....	16
			1.2.5.3 Result and Discussion.....	16
		1.2.6	A lex program to identify floating point numbers.....	17
			1.2.6.1 Problem Description.....	17
			1.2.6.2 Program Code.....	17
			1.2.6.3 Result and Discussion.....	17
		1.2.7	A lex program to identify exponential numbers.....	18
			1.2.7.1 Problem Description.....	18
			1.2.7.2 Program Code.....	18
			1.2.7.3 Result and Discussion.....	18
		1.2.8	A lex program to identify “to be” verb.....	19
			1.2.8.1 Problem Description.....	19
			1.2.8.2 Program Code.....	19
			1.2.8.3 Result and Discussion.....	19
		1.2.9	A lex program to identify a complex number.....	20
			1.2.9.1 Problem Description.....	20
			1.2.9.2 Program Code.....	20
			1.2.9.3 Result and Discussion.....	20
		1.2.10	A lex program to recognize whether a given sentence is simple or compound or complex.....	21
			1.2.10.1 Problem Description.....	21

		1.2.10.2	Program Code.....	21
		1.2.10.3	Result and Discussion.....	21
Chapter 2	YACC			
	2.1	Introduction.....		23
		2.3.1	Overview of Yacc/Bison	24
		2.3.2	Syntax Analysis with Yacc/Bison.....	25
		2.3.3	How to use Yacc/Bison.....	26
	2.2	Example of Yacc/Bison.....		
		2.2.1	Implement YACC program to recognize a valid variable, which starts with a letter, followed by any number of letters or digits.....	27
			2.2.1.1 Problem Description	27
			2.2.1.2 Program Code.....	27
			2.2.1.3 Result and Discussion.....	28
		2.2.2	Implement YACC program to recognize strings 'aaab', 'abbb', 'ab' and 'a' using the grammar ($a^n b^n$, $n>0$).....	29
			2.2.2.1 Problem Description.....	29
			2.2.2.2 Program Code.....	29
			2.2.2.3 Result and Discussion.....	30
		2.2.3	YACC program to recognize a valid arithmetic expression that uses operators +, -, * and /.....	31
			2.2.3.1 Problem Description	31
			2.2.3.2 Program Code.....	31
			2.2.3.3 Result and Discussion.....	32
Chapter 3	ANTLR			
	3.1	Introduction.....		33
		3.1.1	Overview of ANTLR.....	33
		3.1.2	Feature of ANTLR.....	34
		3.1.3	Environment Setting.....	35
	3.2	Example of ANTLR.....		38
		3.2.1	Title of Problem.....	38
			3.2.1.1 Problem Description.....	38
			3.2.1.2 Program Code.....	38
			3.2.1.3 Result and Discussion.....	39
Chapter 4	PROLOG			
	4.1	Introduction.....		40
	4.2	Examples.....		40
		4.2.1	A program to determine if the input is one a followed by one or more b's followed by a single c.....	40
			4.1.1.1 Problem Description.....	40
			4.1.1.2 Algorithm.....	41
			4.1.1.3 Program Code.....	41
			4.1.1.4 Result and Discussion.....	41
		4.2.2	A program to recognize an email address.....	42
			4.2.2.1 Problem Description.....	42
			4.2.2.2 Algorithm.....	42
			4.2.2.3 Program Code.....	42
			4.2.2.4 Result and Discussion.....	43
Chapter 5	PARSER AND LEXER: C/JAVA			

	5.1	Introduction.....	44
	5.2	Examples.....	44
	5.2.1	Construct a program for how to calculate FIRST () & FOLLOW () symbol for LL (1) grammar.....	44
	5.2.1.1	Problem Description.....	44
	5.2.1.2	Algorithm.....	44
	5.2.1.3	Program Code.....	45
	5.2.1.4	Result and Discussion.....	47
	5.2.2	Construct a program to calculate cost of a given instruction.....	48
	5.2.2.1	Problem Description.....	48
	5.2.2.2	Algorithm.....	48
	5.2.2.3	Program Code.....	48
	5.2.2.4	Result and Discussion.....	50
	5.2.3	A program to remove left recursion from a given grammar.....	51
	5.2.3.1	Problem Description.....	51
	5.2.3.2	Algorithm.....	52
	5.2.3.3	Program Code.....	52
	5.2.3.4	Result and Discussion.....	54
		Acknowledgement	55

CHAPTER 01

1. 1. INTRODUCTION

A compiler or an interpreter performs its task in 3 stages:

1) Lexical Analysis:

Lexical analyzer: scans the input stream and converts sequences of characters into tokens.

Token is a classification of groups of characters.

Examples: Lexeme	Token
Sum	ID
for	FOR
:=	ASSIGN_OP
=	EQUAL_OP
57	INTEGER_CONST
*	MULT_OP
,	COMMA
(	LEFT_PAREN

Lex is a tool for writing lexical analyzers.

2) Syntactic Analysis (Parsing):

Parser: reads tokens and assembles them into language constructs using the grammar rules of the language.

Yacc is a tool for constructing parsers.

3) Actions:

Acting upon input is done by code supplied by the compiler writer.

Lex: reads a specification file containing regular expressions and generates a C routine that performs lexical analysis.

Yacc: reads a specification file that codifies the grammar of a language and generates a parsing routine.

1.1.1. OVERVIEW OF FLEX

The first phase in a compiler reads the input source and converts strings in the source to tokens. Using regular expressions, we can specify patterns to lex that allow it to scan and match strings in the input. Each pattern in lex has an associated

action. Typically an action returns a token, representing the matched string, for subsequent use by the parser. To begin with, however, we will simply print the matched string rather than return a token value. We may scan for identifiers using the regular expression

```
letter(letter|digit)*
```

This pattern matches a string of characters that begins with a single letter, and is followed by zero or more letters or digits. This example nicely illustrates operations allowed in regular expressions:

- Repetition, expressed by the “*” operator
- Alternation, expressed by the “|” operator
- Concatenation

1.1.2. LEXICAL ANALYSIS OF FLEX

Flex takes a set of descriptions of possible tokens and produces a scanner. It takes as its input a text file containing regular expressions, together with the action to be taken when each expression is matched. It produces an output file that contains C source code

defining a function `yylex` that is a table-driven implementation of a DFA corresponding to the regular expressions of the input file. The Flex output file is then compiled with a C compiler to get an executable.

As shown below, a lexical specification file for Flex consists of three parts divided by a singline starting with `%%`:

Definitions

`%%`

Rules

`%%`

User Code

In all parts of the specification comments of the form `/* comment text */` are permitted.

1.1.2.1 Definitions

The definition section occurs before the first `%%`. It contains two things. First, any C code that must be inserted external to any function should appear in this section

between the delimiters `%{` and `%}`. Secondly, the definitions section contains declarations of simple name definitions to simplify the scanner specification, and declarations of start conditions. (For a discussion of start conditions, see the Flex Manual ([1]), pages 13 - 18.) Name definitions have the form: name definition

The "name" is a word beginning with a letter or an underscore (

) followed by zero or more

letters, digits, `' '`, or `'-'` (dash). The definition is taken to begin at the first non-white-space character following the name and continuing to the end of the line. The definition can subsequently

be referred to by using "name", which will expand to "(definition)". For example,

`DIGIT [0-9]`

`ID [a-z][a-z0-9]`

*

defines "DIGIT" to be a regular expression which matches a single digit, and "ID" to be a regular expression which matches a letter followed by zero-or-more letters-or-digits.

1.1.2.2 Rules

The "lexical rules" section of a Flex specification contains a set of regular expressions and actions (C code) that are executed when the scanner matches the associated regular expression.

It is of the form:

pattern action where the pattern must be unindented and the action must begin on the same line.

1.1.2.3 Usercode/Auxiliary routines

The user code section is simply copied to "lex.yy.c" (output file generated by Flex) verbatim.

It is used for companion routines which call or are called by the scanner. The presence of this

section is optional; if it is missing, the second `'%%'` in the input file may be skipped, too.

1.1.2.4 Insertion of C Code

1. Any text written between `%{` and `%}` in the definition section will be copied directly to the output program external to any procedure.

2. Any text in the auxiliary procedures section will be copied directly to the output program at the end of the Flex code.

3. Any code that follows a regular expression (by at least one space) in the action section (after the first `%%`) will be inserted at the appropriate place in the

recognition procedure `yylex` and will be executed when a match of the corresponding regular expression occurs.

4. In the rules section, any indented or `%{}` text appearing before the first rule may be used to declare variables which are local to the scanning routine. Other indented or `%{}` text in the rule section is still copied to the output, but its meaning is not well-defined and it may well cause compile-time errors.

1.1.2.5 A brief discussion of start conditions

Start conditions is a mechanism of Flex that enables the use of conditional activation rules.

In this section we will illustrate start conditions by discussing a small example. For further information, consult [1], pages 13-18.

Say, for example, we want a program that replaces a string in a file with the word `string`.

In other words, as soon as we encountered a quotation mark, we want to remove all the text until we find the next quotation mark, and replaced the removed text with the word `string`.

Here is a fragment of code that will accomplish that.

```
1.  
2. %x STRING  
3.  
4. %%  
5.  
6. \" {printf (" string "); BEGIN STRING ;}  
7. <STRING> [^"];  
8. <STRING>\" {BEGIN INITIAL ;}
```

On line 2, we define a start condition `STRING`. On line 6, we define a rule that is applicable if the lexer finds a quotation mark. The action of this rule will enable all rules that start with the prefix `STRING`. In our example it implies that the rules on lines 7 and 8 are enabled. The rule on line 7 matches everything until it finds a quotation mark.

1.1.3. HOW TO USE FLEX

The program `Lex` generates a so called 'Lexer'. This is a function that takes a stream of characters as its input, and whenever it sees a group of characters that match a key, takes a certain action. A very simple example is given in the next page:

```
%{
#include <stdio.h>
}%

%%
stop  printf("Stop command received\n");
start printf("Start command received\n");
%%
```

The first section, in between the %{ and %} pair is included directly in the output program. We need this, because we use printf later on, which is defined in stdio.h.

Sections are separated using '%%', so the first line of the second section starts with the 'stop' key. Whenever the 'stop' key is encountered in the input, the rest of the line (a printf() call) is executed.

Besides 'stop', we've also defined 'start', which otherwise does mostly the same.

We terminate the code section with '%%' again.

To compile Example 1, do this:

```
lex example1.l
cc lex.yy.c -o example1 -ll
```

NOTE: If you are using flex, instead of lex, you may have to change '-ll' to '-lfl' in the compilation scripts. RedHat 6.x and SuSE need this, even when you invoke 'flex' as 'lex'!

This will generate the file 'example1'. If you run it, it waits for you to type some input. Whenever you type something that is not matched by any of the defined keys (ie, 'stop' and 'start') it's output again. If you enter 'stop' it will output 'Stop command received';

Terminate with a EOF (^D).

You may wonder how the program runs, as we didn't define a main() function. This function is defined for you in libl (liblex) which we compiled in with the -ll command.

1.2. EXAMPLES

1.2.1. TITLE OF PROBLEM: A lex program to recognize and count the number of identifier in a given input.

1.2.1.1. PROBLEM DESCRIPTION: Regular expression for identifier in Flex:

```
[a-zA-Z]
[a-zA-Z_][a-zA-Z0-9_]*      identifier: allows a, aX, a45__
```

The user must supply a lexical analyzer to read the input stream and communicate tokens (with values, if desired) to the parser. The lexical analyzer is an integer-valued function called `yylex`. The function returns an integer, the token number, representing the kind of token read. If there is a value associated with that token, it should be assigned to the external variable `yylval`.

The parser and the lexical analyzer must agree on these token numbers in order for communication between them to take place. The numbers may be chosen by Yacc, or chosen by the user. In either case, the ``# define'' mechanism of C is used to allow the lexical analyzer to return these numbers symbolically. For example, suppose that the token name `DIGIT` has been defined in the declarations section of the Yacc specification file. The relevant portion of the lexical analyzer might look like:

```
yylex(){
    extern int yyval;
    int c;
    ...
    c = getchar();
    ...
    switch( c ) {
        ...
        case '0':
        case '1':
            ...
        case '9':
            yyval = c-'0';
            return( DIGIT );
            ...
    }
    ...
}
```

The intent is to return a token number of `DIGIT`, and a value equal to the numerical value of the digit. Provided that the lexical analyzer code is placed in the programs section of the token `DIGIT`.

1.2.1.2. PROGRAM CODE:

```
%{  
#include<stdio.h>  
%}  
%%  
id [a-zA-Z]+  
id[a-zA-Z0-9]+  
%%  
{id} {printf("\n %s is an identifier\n",yytext);}  
  
int main()  
{  
printf("Enter the expression\n");  
yylex();  
return 0;  
}
```

1.2.1.3. RESULT AND DISCUSSION:

OUTPUT

```
$lex p2a.l  
$cc lex.yy.c -ll  
$./a.out  
Enter the expression  
(a+b*c)  
a is an identifier  
b is an identifier  
c is an identifier
```

1.2.2. TITLE OF PROBLEM: A lex program to count the characters, words, spaces and lines in a given input file.

1.2.2.1. PROBLEM DESCRIPTION: The main task of the lexical analyzer is to read the input source program, scanning the characters, and produce a sequence of tokens that the parser can use for syntactic analysis.

The interface may be to be called by the parser to produce one token at a time. Maintain internal state of reading the input program (with lines). Have a function “getNextToken” that will read some characters at the current state of the input and return a token to the parser. Other tasks of the lexical analyzer include Skipping or hiding whitespace and comments. Keeping track of line numbers for error reporting. Sometimes it can also produce the annotated lines for error reports that produce the value of the token. Optional: Insert identifiers into the symbol table.

1.2.2.2. PROGRAM CODE:

```
%{
int ch=0, bl=0, ln=0, wr=0;
}%

%%
[\n] {ln++;wr++;}
[\t] {bl++;wr++;}
[" "] {bl++;wr++;}
[^\\n\\t] {ch++;}
%%

int main()
{
FILE *fp;
char file[10];
printf("Enter the filename: ");
scanf("%s", file);
yyin=fp;
yylex();
printf("Character=%d\\nBlank=%d\\nLines=%d\\nWords=%d", ch, bl, ln, wr);
return 0;
}
```

1.2.2.3. RESULT AND DISCUSSION:

INPUT

A input file (.doc or any format), counts number of characters, words, spaces and Lines in a

given input file.

OUTPUT

```
$cat > input
```

```
Girish rao salanke
```

```
$lex p1a.l
```

```
$cc lex.yy.c -ll
```

```
$/a.out
```

```
Enter the filename: input
```

```
Character=16
```

```
Blank=2
```

```
Lines=1
```

```
Word=3
```

1.2.3. TITLE OF PROBLEM: A lex program to count the numbers of comment lines in a given C/C++/JAVA program. Also eliminate them and copy the resulting program into separate file.

1.2.3.1. PROBLEM DESCRIPTION: The character stream input is grouped into meaningful units called **lexemes**, which are then mapped into **tokens**, the latter constituting the output of the lexical analyzer. For example, any one of the following

```
x3 = y + 3;  
x3 = y + 3 ;  
x3 =y+ 3 ;
```

but not

```
x 3 = y + 3;
```

would be grouped into the lexemes x3, =, y, +, 3, and ;.

A token is a <token-name, attribute-value> pair. For example

1. The lexeme x3 would be mapped to a token such as <id, 1>. The name id is short for identifier. The value 1 is the index of the entry for x3 in the symbol table produced by the compiler. This table is used to pass information to subsequent phases.

2. The lexeme = would be mapped to the token <=>. In reality it is probably mapped to a pair, whose second component is ignored. The point is that there are many different identifiers so we need the second component, but there is only one assignment symbol

=.

3. The lexeme y is mapped to the token <id, 2>

4. The lexeme + is mapped to the token <+>.

5. The lexeme 3 is somewhat interesting and is discussed further in subsequent chapters. It is mapped to <number, something>, but what is the something. On the one hand there is only one 3 so we could just use the token <number, 3>. However, there can be a difference between how this should be printed (e.g., in an error message produced by subsequent phases) and how it should be stored (fixed vs. float vs double). Perhaps the token should point to the symbol table where an entry for this kind of 3 is stored. Another possibility is to have a separate numbers table.

6. The lexeme ; is mapped to the token <;>.

Note that non-significant blanks are normally removed during scanning. In C, most blanks are non-significant. Blanks inside strings are an exception.

Note that we can define identifiers, numbers, and the various symbols and punctuation without using recursion (compare with parsing below).

1.2.3.3: PROGRAM CODE:

```
%{
int com=0;
}%
%%
"/*"^\n]+""/" {com++;fprintf(yyout, " ");}
%%
int main()
{
printf("Write a C program\n");
yyout=fopen("output", "w");
yylex();
printf("Comment=%d\n",com);
return 0;
}
```

1.2.3.4: RESULT AND DISCUSSION:

```
$lex p1b.l
$cc lex.yy.c -ll
$./a.out
Write a C program
#include<stdio.h>
int main()
{
    int a, b;
    /*float c;*/
    printf("Hi");
    /*printf("Hello");*/
}
[Ctrl-d]
Comment=1
$cat output
#include<stdio.h>
int main()
{
    int a, b;
    printf("Hi");
}
```

1.2.4. TITLE OF PROBLEM: A lex program to identify an integer number.

1.2.4.1. PROBLEM DESCRIPTION: The user must supply a lexical analyzer to read the input stream and communicate tokens (with values, if desired) to the parser. The lexical analyzer is an integer-valued function called yylex. The function returns an integer, the token number, representing the kind of token read. If there is a value associated with that token, it should be assigned to the external variable yylval.

Regular expression for only integer number in Flex:

`[0-9]+`

1.2.4.2. PROGRAM CODE:

```
%{
#include<stdio.h>
%}
%%
integer[0-9]+
%%
{integer} {printf("\n %s is an integer\n",yytext);}
%%
int main()
{
printf("Enter the number.\n");
yylex();
return 0;
}
```

1.2.4.3. RESULT AND DISCUSSION:

```
$lex pa.l
$cc lex.yy.c -ll
$/a.out
Enter the number.
(1+2*3)
1 is an integer
2 is an integer
3 is an integer
```

1.2.5. TITLE OF PROBLEM: A lex program to identify a Teletalk number.

1.2.5.1. PROBLEM DESCRIPTION:

The user must supply a lexical analyzer to read the input stream and communicate tokens (with values, if desired) to the parser. The lexical analyzer is an integer-valued function called yylex. The function returns an integer, the token number, representing the kind of token read. If there is a value associated with that token, it should be assigned to the external variable yylval. Regular expression for teletalk number in Flex:

(015) [0-9] [0-9] [0-9] [0-9] [0-9] [0-9] [0-9] [0-9]

1.2.5.2. PROGRAM CODE:

```
%{
#include<stdio.h>
%}
%%
(015) [0-9] [0-9] [0-9] [0-9] [0-9] [0-9] [0-9] [0-9] {printf("This is a teletalk number.");}
[0-9]+ {printf("This is not a teletalk number")}
.|\\n
{ECHO;}
%%
main(){
yylex();
}
```

1.2.5.3. RESULT AND DISCUSSION:

```
$lex pa.l
$cc lex.yy.c -ll
$./a.out
Enter the expression
01520090569
This is a teletalk number.
```

1.2.6. TITLE OF PROBLEM: A lex program to identify floating point numbers.

1.2.6.1. PROBLEM DESCRIPTION: The user must supply a lexical analyzer to read the input stream and communicate tokens (with values, if desired) to the parser. The lexical analyzer is an integer-valued function called yylex. The function returns an integer, the token number, representing the kind of token read. If there is a value associated with that token, it should be assigned to the external variable yylval.

Regular expression for floating point number in Flex:

`[0-9]+\".\"[0-9]+`

1.2.6.2. PROGRAM CODE:

```
%{
#include<stdio.h>
%}
float [0-9]+\".\"[0-9]
%%
{float} {printf ("\n %s is a floating point number.\n", yytext);}
%%
int main()
{
printf("Enter the number.\n");
yylex();
return 0;
}
```

1.2.6.3. RESULT AND DISCUSSION:

```
$lex pa.l
$cc lex.yy.c -ll
$./a.out
Enter the number.
2.55
```

2.55 is a floating point number.

1.2.7. TITLE OF PROBLEM: A lex program to identify exponential numbers.

1.2.7.1. PROBLEM DESCRIPTION: The user must supply a lexical analyzer to read the input stream and communicate tokens (with values, if desired) to the parser. The lexical analyzer is an integer-valued function called `yylex`. The function returns an integer, the token number, representing the kind of token read. If there is a value associated with that token, it should be assigned to the external variable `yyval`.

1.2.7.2. PROGRAM CODE:

```
%{  
# include <stdio.h>  
%}  
I [0-9]+  
%%  
[eE][+-]? {I} {printf("%s is an exponential number.", yytext);}  
%%  
main ()  
{  
    yylex();  
}
```

1.2.7.3. RESULT AND DISCUSSION:

```
$lex pa.l  
$cc lex.yy.c -ll  
$./a.out  
Enter the number.  
7e7  
  
7e7 is an exponential number.
```

1.2.8. TITLE OF PROBLEM: A lex program to identify “to be” verb.

1.2.8.1. PROBLEM DESCRIPTION: The user must supply a lexical analyzer to read the input stream and communicate tokens (with values, if desired) to the parser. The lexical analyzer is an integer-valued function called yylex. The function returns an integer, the token number, representing the kind of token read. If there is a value associated with that token, it should be assigned to the external variable yylval.

1.2.8.2. PROGRAM CODE:

```
%{
#include<stdio.h>
%}
verb [am][is][are][was][were][being][been][be]
%%
{float} {printf ("\n %s is a “to be” verb.\n", yytext);}
%%
int main()
{
printf("Enter the verb.\n");
yylex();
return 0;
}
```

1.2.8.3. RESULT AND DISCUSSION:

```
$lex pa.l
$cc lex.yy.c -ll
$./a.out
Enter the verb.
was
was is a “to be” verb.
```

1.2.9. TITLE OF PROBLEM: A lex program to identify a complex number.

1.2.9.1. PROBLEM DESCRIPTION: The user must supply a lexical analyzer to read the input stream and communicate tokens (with values, if desired) to the parser. The lexical analyzer is an integer-valued function called yylex. The function returns an integer, the token number, representing the kind of token read. If there is a value associated with that token, it should be assigned to the external variable yylval.

1.2.9.2. PROGRAM CODE: %{\n#include<stdio.h>\n%\ncom [i]”+”[0-9]\ncom[i]”-”[0-9]\n%%\n{float} {printf (“\\n %s is a complex number.\\n”, yytext);} \n%%\nint main()\n{\nprintf(“Enter the number.\\n”);\nyylex();\nreturn 0;\n}

1.2.9.3. RESULT AND DISCUSSION:

```
$lex pa.l
$cc lex.yy.c -ll
$./a.out
Enter the number.
i+2
```

i+2 is a floating point number

1.2.10. TITLE OF PROGRAM: A lex program to recognize whether a given sentence is simple or compound or complex.

1.2.10.1. PROBLEM DESCRIPTION: The user must supply a lexical analyzer to read the input stream and communicate tokens (with values, if desired) to the parser. The lexical analyzer is an integer-valued function called yylex. The function returns an integer, the token number, representing the kind of token read. If there is a value associated with that token, it should be assigned to the external variable yylval.

1.2.10.2. PROGRAM CODE:

```
%{
int flag=0;
}%
%%
("[aA][nN][dD]"|("[oO][rR]"|("[bB][uU][tT]" {flag=1;}
("[sS][iI][nN][cC][eE]"|("[aA][sS]"|("[wW][hH][eE][nN]" {flag=2;}
%%
int main()
{
printf("Enter the sentence\n");
yylex();
if(flag==1)
printf("\nCompound sentence\n");
elseif(flag==2)
printf("\nComplex sentence\n");
else
printf("\nSimple sentence\n");
return 0;
}
```

2.2.4.3. RESULT AND DESCRIPTION:

```
$lex p2b.l
$cc lex.yy.c -ll
$./a.out
Enter the sentence
I am Arnisha
I am Arnisha
```

[Ctrl-d]

Simple sentence

\$/a.out

Enter the sentence

CSE or ISE

CSE or ISE

[Ctrl-d]

Compound sentence

\$/a.out

Enter the sentence

When it rains, I feel dizzy

[Ctrl-d]

Complex sentence

CHAPTER 2

PARSER

2.1 INTRODUCTION

A parser is a software component that takes input data (frequently text) and builds a data structure – often some kind of parse tree, abstract syntax tree or other hierarchical structure – giving a structural representation of the input, checking for correct syntax in the process. The parsing may be preceded or followed by other steps, or these may be combined into a single step. The parser is often preceded by a separate lexical analyzer, which creates tokens from the sequence of input characters; alternatively, these can be combined in scanner less parsing. Parsers may be programmed by hand or may be automatically or semi-automatically generated by a parser generator. Parsing is complementary to templating, which produces formatted *output*. These may be applied to different domains, but often appear together, such as the `scanf/printf` pair, or the input (front end parsing) and output (back end code generation) stages of a compiler.

The input to a parser is often text in some computer language, but may also be text in a natural language or less structured textual data, in which case generally only certain parts of the text are extracted, rather than a parse tree being constructed. Parsers range from very simple functions such as `scanf`, to complex programs such as the frontend of a C++ compiler or the HTML parser of a web browser. An important class of simple parsing is done using regular expressions, where a regular expression defines a regular language, and then the regular expression engine automatically generates a parser for that language, allowing pattern matching and extraction of text. In other contexts regular expressions are instead used prior to parsing, as the lexing step whose output is then used by the parser.

The use of parsers varies by input. In the case of data languages, a parser is often found as the file reading facility of a program, such as reading in HTML or XML text; these examples are markup languages. In the case of programming languages, a parser is a component of a compiler or interpreter, which parses the source code of a computer programming language to create some form of internal representation; the parser is a key step in the compiler frontend. Programming languages tend to be specified in terms of a deterministic context-free grammar because fast and efficient parsers can be written for them. For compilers, the parsing itself can be done in one pass or multiple passes – see one-pass compiler and multi-pass compiler.

2.1.1. OVERVIEW OF YACC

Yacc specification describes a CFG that can be used to generate a parser.

Elements of a CFG:

1. Terminals: tokens and literal characters,
2. Variables (nonterminals): syntactical elements,
3. Production rules, and
4. Start rule.

Format of a production rule:

```
symbol:    definition
           {action}
           ;
```

Example:

$A \rightarrow Bc$ is written in yacc as `a: b 'c';`

Not all grammars are LR (1). Occasionally, when building the LR parse table, yacc will find duplicate entries.

There are two kinds of duplicate entries – shift-reduce conflicts and reduce-reduce conflicts. We will look at each in turn

Shift-Reduce Conflicts

Shift-reduce conflicts arise when yacc can't determine if a token should be shifted, or if a reduce should be done instead. The canonical example of this problem is the dangling else. Consider the following two rules for an if statement:

(1) $S \rightarrow \text{if } (E) S \text{ else } S$

(2) $S \rightarrow \text{if } (E) S$

Now consider the following sequence of tokens: `if (id) if (id) print (id) else print(id)`

What happens when we hit that else? Should we assume that the interior if statement has no else, a reduce by rule (2)? Or should we assume that the interior if statement does have an else, and shift, with thintent of reducing the interior if statement using rule (1)? If it turns out that both ifs have elses, we'll be trouble if we reduce now. What if only one of the ifs has an else? We'll be OK either way. Thus the correanswer is to shift in this case. If we have an if statement like `if (<test>) if (<test>) <statement>else <statement>`, shifting instead of reducing on the else will cause the else to be bound to the innermoif. That's another good reason for resolving the dangling if by having the else bind to the inner statement writing a parser is easier!

Thus, whenever yacc has the option to either shift or reduce, it will always shift. A warning will bproduced (since other than the known dangling-else problem, shift-reduce conflicts are almost always signerrors in your grammar), but the parse table will be created.

Reduce-Reduce Conflicts

Consider the following simple grammar:

- (1) $S \rightarrow A$
- (2) $S \rightarrow B$
- (3) $A \rightarrow ab$
- (4) $B \rightarrow ab$

What should an LR parser of this grammar do with the file:

First, the a and b are shifted. Then what? Should the ab on the stack be reduced to an A by rule 3), or should the ab be reduced to a B by rule (4)? Should an r(3) or an r(4) entry appear in the table? Reduce-reduce conflicts are typically more serious than shift-reduce conflicts. A reduce-reduce conflict is most always a sign of an error in the grammar. Yacc resolves reduce-reduce conflict by using the first rule that appears in the .grm file. So, in the above example, the table would contain an r(3) entry. However, when using yacc, never count on this behavior for reduce-reduce conflicts! Instead, rewrite the grammar to void the conflict. Just because yacc produces code on an reduce-reduce conflict does not mean that the code does what we want!

2.1.2. SYNTAX ANALYSIS WITH Yacc

Yacc (Yet Another Compiler Compiler) is a program designed to compile a LALR(1) grammar and to produce the source code of the syntactical analyser of the language produced by this grammar. It is also possible to make it do semantic actions. As for a Lex file, a Yacc file can be divided into three parts:

declarations

%%

productions

%%

additional code

and only the first %% and the second part are mandatory.

2.1.2.1 The first part of a Yacc file

The first part of a Yacc file may contain:

- Specifications written in the target language, enclosed between %{ and %} (each symbol at the beginning of a line) that will be put at the top of the scanner generated by Yacc.
- Declaration of the tokens that can be encountered:
%token TOKEN
- The type of the terminal, using the reserved word: %union

- Informations about operators' priority or associativity.
- The axiom of the grammar, using the reserved word %start (if not specified, the axiom is the first production of the second part of the file).

The yylval variable, implicitly declared of the %union type is really important in the file, since it is the variable that contains the description of the last token read.

2.1.2.2 Second part of a Yacc file

This part cannot be empty. It may contain:

- Declarations and/or definitions enclosed between %{ and %}.
- Productions of the language's grammar. They look like:

nonterminal_notion:

```

        body_1 {semantical_action_1 }
        | body_2 {semantical_action_2 }
        | ...
        | body_n { semantical_action_n }
        ;

```

provided that the "body i" may be terminal or nonterminal notions of the language.

2.1.2.3 Third part of a Yacc file

This part contains the additional code, must contain a main () function (that should call the yyparse() function), and an yyerror(char *message) function, that is called when a syntax error is found.

2.1.2.4 Conclusion about Yacc

This presentation is far from being exhaustive, and I didn't explain you everything. We will clarify some points in the following example.

2.1.3. HOW TO USE Yacc

Provided that the Lex file is called calc.lex, and the Yacc file calc.y, all we have to do is:

```

>bison -d calc.y
>mv calc.tab.h calc.h
>mv calc.tab.c calc.y.c
>flex calc.lex
>mv lex.yy.c calc.lex.c
>gcc -c calc.lex.c -o calc.lex.o
>gcc -c calc.y.c -o calc.y.o
>gcc -o calc calc.lex.o calc.y.o -lfl -lm [eventually -ll]

```

2.2. EXAMPLES

2.2.1. TITLE OF PROBLEM: Implement YACC program to recognize a valid variable, which starts with a letter, followed by any number of letters or digits.

2.2.1.1. PROBLEM DESCRIPTION: Yacc turns the specification file into a C program, which parses the input according to the specification given. The algorithm used to go from the specification to the parser is complex, and will not be discussed here (see the references for more information). The parser itself, however, is relatively simple, and understanding how it works, while not strictly necessary, will nevertheless make treatment of error recovery and ambiguities much more comprehensible.

2.2.1.2. PROGRAM CODE:

```
LEX
%{
#include"y.tab.h"
extern yylval;
}%
%%
[0-9]+ {yylval=atoi(yytext); return DIGIT;}
[a-zA-Z]+ {return LETTER;}
[\t] ;
\n return 0;
. {return yytext[0];}
%%
YACC
%{
#include<stdio.h>
}%
%token LETTER DIGIT
%%
variable: LETTER|LETTER rest
;
rest: LETTER rest
;
%%
|DIGIT rest
|LETTER
|DIGIT
main()
{
yyparse();
```

```

printf("The string is a valid variable\n");
}
int yyerror(char *s)
{
printf("this is not a valid variable\n");
exit(0);
}

```

2.2.1.3. RESULT AND DISCUSSION:

```

$lex p4b.l
$yacc -d p4b.y
$cc lex.yy.c y.tab.c -ll
$./a.out
input34
The string is a valid variable
$./a.out
89file
This is not a valid variable

```

2.2.2. TITLE OF PROBLEM: Implement YACC program to recognize strings 'aaab', 'abbb', 'ab' and 'a' using the grammar ($a^n b^n$, $n > 0$).

2.2.2.1. PROBLEM DESCRIPTION: Yacc provides a general tool for imposing structure on the input to a computer program. The Yacc user prepares a specification of the input process; this includes rules describing the input structure, code to be invoked when these rules are recognized, and a low-level routine to do the basic input. Yacc then generates a function to control the input process. This function, called a parser, calls the user-supplied low-level input routine (the lexical analyzer) to pick up the basic items (called tokens) from the input stream. These tokens are organized according to the input structure rules, called grammar rules; when one of these rules has been recognized, then user code supplied for this rule, an action, is invoked; actions have the ability to return values and make use of the values of other actions.

Yacc is written in a portable dialect of C and the actions, and output subroutine, are in C as well. Moreover, many of the syntactic conventions of Yacc follow C.

2.2.2.2. PROGRAM CODE:

```
LEX
%{
#include"y.tab.h"
%}
%%
[a] return A;
[b] return B;
%%
YACC
%{
#include<stdio.h>
%}
%token A B
%%
S:A S B
|
;
%%
main ()
{
printf("Enter the string\nif(yyparse()==0)
{
printf ("Valid\n");
}
}
yyerror(char *s)
```

```
{  
printf("%s\n",s);  
}
```

2.2.2.3. RESULT AND DISCUSSION:

```
$lex p5b.l  
$yacc -d p5b.y  
$cc lex.yy.c y.tab.c -ll  
$./a.out  
Enter the string  
aabb  
[Ctrl-d]  
Valid  
$./a.out  
Enter the string  
aab  
syntax error
```

2.2.3. TITLE OF PROGRAM: YACC program to recognize a valid arithmetic expression that uses operators +, -, *, and /.

2.2.3.1. PROBLEM DESCRIPTION: Names refer to either tokens or nonterminal symbols. Yacc requires token names to be declared as such. It is often desirable to include the lexical analyzer as part of the specification file; it may be useful to include other programs as well. Thus, every specification file consists of three sections: the declarations, (grammar) rules, and programs. The sections are separated by double percent ``%%" marks.

(The percent ``%" is generally used in Yacc specifications as an escape character.)

In other words, a full specification file looks like

declarations

%%

rules

%%

programs

The declaration section may be empty. Moreover, if the programs section is omitted, the second %% mark may be omitted also; thus, the smallest legal Yacc specification is

%%

rules

2.2.3.2. PROGRAM CODE:

LEX

%{

#include"y.tab.h"

extern yylval;

%}

%%

[0-9]+ {yylval=atoi(yytext); return NUMBER;}

[a-zA-Z]+ {return ID ;}

[\t]+ ;

\n {return 0;}

. {return yytext[0];}

%%

YACC

%{

#include<stdio.h>

%}

%token NUMBER ID

%left '+' '-'

%left '*' '/'

%%

```

expr: expr '+' expr
    | expr '-' expr
    | expr '*' expr
    | expr '/' expr
    | '-'NUMBER
    | '-'ID
    | '('expr')'
    | NUMBER
    | ID
    ;
%%
main ()
{
printf("Enter the expression\n");
yyvsparse();
printf("\nExpression is valid\n");
exit(0);
}
int yyerror(char *s)
{
printf("\nExpression is invalid");
exit(0);
}

```

2.2.3.3. RESULT AND DISCUSSION:

```

$lex p4a.l
$yacc -d p4a.y
$cc lex.yy.c y.tab.c -ll
$./a.out
Enter the expression
(a*b+5)
Expression is valid
$./a.out
Enter the expression
(a+6-)
Expression is invalid

```

CHAPTER 3 PARSER: ANTLR

3.1 INTRODUCTION

ANTLR (ANother Tool for Language Recognition) is a so called parser generator, or, sometimes called a compiler-compiler. In a nutshell: given a grammar of a language, ANTLR can generate a lexer and parser for said language. It is a powerful parser generator for reading, processing, executing, or translating structured text or binary files. It's widely used to build languages, tools, and frameworks. From a grammar, ANTLR generates a parser that can build and walk parse trees.

3.1.1. OVERVIEW OF ANTLR

In computer-based language recognition, ANTLR (pronounced Antler), or ANother Tool for Language Recognition, is a parser generator that uses LL(*) parsing. ANTLR is the successor to the Purdue Compiler Construction Tool Set (PCCTS), first developed in 1989, and is under active development. Its maintainer is professor Terence Parr of the University of San Francisco.

ANTLR takes as input a grammar that specifies a language and generates as output source code for a recognizer for that language. While version 3 supported generating code in the programming languages Ada95, Action Script, C, C#, Java, JavaScript, Objective-C, Perl, Python, Ruby, and Standard ML, the current release at present only targets Java and C#. A language is specified using a context-free grammar which is expressed using Extended Backus–Naur Form (EBNF).

ANTLR can generate lexers, parsers, tree parsers, and combined lexer-parsers. Parsers can automatically generate abstract syntax trees which can be further processed with tree parsers. ANTLR provides a single consistent notation for specifying lexers, parsers, and tree parsers. This is in contrast with other parser/lexer generators and adds greatly to the tool's ease of use.

By default, ANTLR reads a grammar and generates a recognizer for the language defined by the grammar (i.e. a program that reads an input stream and generates an error if the input stream does not conform to the syntax specified by the grammar). If there are no syntax errors, then the default action is to simply exit without printing any message. In order to do something useful with the language, actions can be attached to grammar elements in the grammar. These actions are written in the programming language in which the recognizer is being generated. When the recognizer is being generated, the actions are embedded in the source code of the recognizer at the appropriate points. Actions can be used to build and check symbol tables and to emit instructions in a target language, in the case of a compiler.

As well as lexers and parsers, ANTLR can be used to generate tree parsers. These are recognizers that process abstract syntax trees which can be automatically

generated by parsers. These tree parsers are unique to ANTLR and greatly simplify the processing of abstract syntax trees.

3.1.2. FEATURES OF ANTLR

ANTLR's popularity comes down to the fact that it satisfies the following fundamental requirements. Programmers want to use tools:

1. that employ mechanisms they understand,
2. that are sufficiently powerful to solve their problem,
3. that are flexible,
4. that automate tedious tasks, and
5. that generate output that is easily folded into their application.

ANTLR has a consistent syntax for specifying lexers, parsers, and tree parsers.

Explaining why tree parsers are useful is difficult until we have some experience building translators; nonetheless, ANTLR is one of the few language tools that lets us apply grammatical structure of trees.

ANTLR generates powerful recognizers with its semantic and syntactic predicates. Plus PCCTS/ANTLR was the first widely-used parser generator to employ $k > 1$ lookahead. By using ANTLR, we can be certain that we are not betting our project on a "dead" tool. Many academic and industry projects use ANTLR.

There are existing grammars available for many languages. ANTLR modes for emacs, Eclipse, and other IDEs are available. ANTLR currently generates Java, C++, C# and Python. ANTLR has pretty flexible and decent error handling.

ANTLR comes with complete source code unlike many other systems and has absolutely no restrictions on its use. I have placed it totally in the public domain.

3.1.3. ENVIRONMENT SETTING:

3.1.3.1. Getting a proper Java virtual machine running: Install a compatible Java virtual machine. We can skip this step if Java is properly installed.

On Ubuntu or Debian Linux, we can install OpenJDK from the package manager:

```
sudo apt-get install default-jdk
```

On other platforms, use either the Oracle/Sun distribution or Open JDK.
Check if Java installation is working:

```
java -version
```

Expect output like the following (for OpenJDK on Ubuntu):

```
java version "1.6.0_20"
```

```
OpenJDK Runtime Environment (IcedTea6 1.9.5) (6b20-1.9.5-0ubuntu1~10.04.1)
```

```
OpenJDK 64-Bit Server VM (build 19.0-b09, mixed mode)
```

CLASS PATH SELECTION:

Start menu> Computer>Advanced system settings>Advanced tab> Environment variables> Path> Edit> modify path.

In the Edit window, modify **PATH** by adding the location of the class to the value for **PATH**. If you do not have the item **PATH**, you may select to add a new variable and add **PATH** as the name and the location of the class as the value.

Reopen Command prompt window, and run java code.

When installing the JDK (Java Development Kit) on Windows 7, the **javac** command does not work in the command prompt. This is because we did not include in our **PATH** environment variable the folder path to the **javac** application of JDK.

Right-click on (My) Computer > Properties> Advanced setting from left pane>Advanced tab> Environment variable> View> Path>Edit> "C:\Program Files\Java\jdk1.6.0_31\bin\"

Then go to the command prompt and type **javac -version**

It should output the current version of your Java compiler.

```
javac 1.6.0_31
```

3.1.3.2. Installing ANTLR: Visit the download page and download the "Complete ANTLR x.y Java binaries jar" file.

- For example, from a Linux shell, download ANTLR 3.3 to home directory:
- `cd ~`

- `wget http://www.antlr.org/download/antlr-3.3-complete.jar`
Add ANTLR to CLASSPATH environmental variable and run it:

```
export CLASSPATH=~/antlr-3.3-complete.jar:$CLASSPATH
java org.antlr.Tool -version
```

Expect output like the following:

```
ANTLR Parser Generator Version 3.3 Nov 30, 2010 12:50:56
```

If we see output like the following, the CLASSPATH is not set up properly:

```
Exception in thread "main" java.lang.NoClassDefFoundError
```

If we see an older version of ANTLR, your CLASSPATH may not be set up properly and Java may be finding ANTLR in .jar files bundled with an application like BEA WebLogic. Ensure the path to the current .jar of ANTLR is at the beginning of our CLASSPATH.

Configure CLASSPATH to include ANTLR on future logins.

- For example, on a BASH shell, add the environmental variable to the .bashrc script:

```
echo "export CLASSPATH=~/antlr-3.3-complete.jar:$CLASSPATH" >>
~/.bashrc
```

3.1.3.3. HOW TO RUN ANTLR:

For windows

```
java -cp antlr-3.2.jar org.antlr.ToolExp.g
```

```
javac -cp .;antlr-3.2.jar ANTLRDemo.java
```

```
java -cp .;antlr-3.2.jar ANTLRDemo
```

For linux

```
java -cp antlr-3.2.jar org.antlr.ToolExp.g
```

```
javac -cp .:antlr-3.2.jar ANTLRDemo.java
```

```
java -cp .:antlr-3.2.jar ANTLRDemo
```

3.2EXAPMLE:

3.2.1. TITLE OF PROBLEM: Using ANTLR compute the result of the following expression $2*3+1$.

3.2.1.1. PROBLEM DESCRIPTION:

Using ANTLR Editor we write our antlr program then save the program using .g extension. Here we save antlr program file as antlr.g. Then we write an ANTLRDemo.java file. We must keep an executable jar file antlr-3.2 in the folder which contains .g program file and ANTLRDemo.java file. Then using run command we can get our desired result.

3.2.1.2. PROGRAM CODE:

Antlr.g:

```
grammar antlr;
eval returns [double value]
    : exp=additionExp {$value = $exp.value;}
    ;
additionExp returns [double value]
    : m1=multiplyExp {$value = $m1.value;}
      ( '+' m2=multiplyExp {$value += $m2.value;}
      | '-' m2=multiplyExp {$value -= $m2.value;}
      )*
    ;
multiplyExp returns [double value]
    : a1=atomExp {$value = $a1.value;}
      ( '*' a2=atomExp {$value *= $a2.value;}
      | '/' a2=atomExp {$value /= $a2.value;}
      )*
    ;
atomExp returns [double value]
    : n=Number {$value = Double.parseDouble($n.text);}
    | '(' exp=additionExp ')' {$value = $exp.value;}
    ;

Number
    : ('0'..'9')+ ('.' ('0'..'9'))?
    ;
WS
    : (' '|'\t'|\r'|\n') {$channel=HIDDEN;}
    ;
ID
    : ('a'..'z'|'A'..'Z')+;
```

ANTLRDemo.java:

```
import java.util.Scanner;
import javax.sound.midi.Soundbank;
import org.antlr.runtime.*;

public class ANTLRDemo {
    public static void main(String[] args) throws Exception {

        System.out.println(" Enter your expression :\n");
        String a=new String();
        Scanner input=new Scanner(System.in);
        a=input.nextLine();
        ANTLRStringStream in = new ANTLRStringStream(a);
        antlrLexer lexer = new antlrLexer(in);
        CommonTokenStream tokens = new CommonTokenStream(lexer);
        antlrParser parser = new antlrParser(tokens);
        System.out.println(parser.eval()); // print the value
    }
}
```

3.1.2.3. RESULT AND DISCUSSION:

For windows, the instructions below are used to run the program:

```
java -cp antlr-3.2.jar org.antlr.ToolExp.g
javac -cp .;antlr-3.2.jar ANTLRDemo.java
java -cp .;antlr-3.2.jar ANTLRDemo
```

After that, the following would happen:

Enter your expression:

2*3+1

7.0

CHAPTER 4 PROLOG

4.1. Introduction

Prolog is a *declarative* programming language. This means that in prolog, we do not write out what the computer should do line by line, as in *procedural* languages such as C and Java. The general idea behind declarative languages is that we describe a situation. Based on this code, the interpreter or compiler will tell us a solution. In the case of prolog, it will tell us whether a prolog sentence is true or not and, if it contains variables, what the values of the variables need to be.

This may sound like a godsend for programmers, but the truth is that prolog is seldom used purely in this way. Though the declarative idea is the backbone of prolog, it is possible to see prolog code as procedural. A prolog programmer will generally do both depending on the part of the code he or she is reading and writing. When learning prolog however, experience in procedural programming is in no way useful (it is often said that it is easier to learn prolog for someone who does not have any experience in procedural programming than for someone who does)

Prolog is considered a difficult language to master, especially when the student tries to rush things, mainly because of the different way of thinking the student has to adopt and amount of recursion in prolog programs. When used correctly, however, prolog can be a very powerful language.

4.2. EXAMPLES

4.2.1. Title of Problem

A program to determine if the input is one a followed by one or more b's followed by a single c.

4.2.1.1. Problem Description

A finite state machine is a quintuple with

- Q a finite set of states
- Σ a finite set of symbols, the alphabet
- $S \subseteq Q$ the set of start states
- $F \subseteq Q$ the set of final states
- E a set of edges

The FSA is represented by the following kind of Prolog facts:

- Initial nodes: initial (*nodename*).
- Final nodes: final (*nodename*).
- Edges: arc (*from-node*, *label*, *to-node*).

4.2.1.2. Algorithm

An automaton starts with 'a' then one or more 'b' and finish with 'c', otherwise not rejected.

4.2.1.3. Program Code

```
Delta(q0, a, q1). Delta(q0, b, q4). Delta(q0, c, q4).  
Delta(q1, a, q4). Delta(q1, b, q2). Delta(q1, c, q4).  
Delta(q2, a, q4). Delta(q2, b, q2). Delta(q2, c, q3).  
Delta(q3, a, q4). Delta(q3, b, q4). Delta(q3, c, q4).  
Delta(q4, a, q4). Delta(q4, b, q4). Delta(q4, c, q4).  
Initial(q0).  
Accepting(q3).
```

```
Recognize(Input):- initial(state0), run(Input, state0, state), accepting(state).  
Run([], state, state).  
Run([I|Is], state0, state):- delta(state0, I, state1), run(Is, state1, state).
```

4.2.1.4. Result and Discussion

Input: Recognize ([abbbc]).

Output: True

Input: Recognize ([babc]).

Output: False

4.2.2. TITLE OF PROBLEM: A program to recognize an email address.

4.2.2.1. PROBLEM DESCRIPTION:

A program for email recognition constitutes a huge amount of code as each transition can be done for each of 26 letters of the English alphabet and each of the

digits from 0-9. Thus, total 36 inputs for each transition are needed for constructing a program that recognizes an email address.

For convenience, in this example, we construct a program code that accepts an email address like a.a@a.com.

4.2.2.2. ALGORITHM:

delta(q0,a,q1).
delta(q1,a,q1).
delta(q1,,q2).
delta(q2,a,q3).
delta(q3,,q2).
delta(q3,@,q4).
delta(q4,a,q5).
delta(q5,,q6).
delta(q6,c,q7).
delta(q7,o,q8).
delta(q8,m,q9).
initial(q0).
accepting(q9).

4.2.2.3. PROGRAM CODE:

delta(q0,a,q1).
delta(q1,a,q1).
delta(q1,,q2).
delta(q2,a,q3).
delta(q3,,q2).
delta(q3,@,q4).
delta(q4,a,q5).
delta(q5,,q6).
delta(q6,c,q7).
delta(q7,o,q8).
delta(q8,m,q9).
initial(q0).
accepting(q9).

recognize(Input):-

```
initial(State0),  
run(Input,State0,State),  
accepting(State),  
run([],State,State).  
run([I|Is],State0,State):-  
delta(State0,I,State1),  
run(Is,State1,State).
```

4.2.2.4. RESULT AND DESCRIPTION:

Input: recognize (a,.,a,@,a,.,c,o,m).

Output: true

Input: recognize (a,.,a,.,a,@,a,.,c,o,m).

Output: false

CHAPTER 5 PARSER AND LEXER: C/ JAVA

5.1. Introduction

5.2. EXAMPLES:

5.2.1. Title of Problem

Construct a program for how to calculate FIRST () & FOLLOW () symbol for LL (1) grammar

5.2.1.1. Problem Description

Recursive descent parsers needing no backtracking can be constructed for a class of grammars called LL (1). The first 'L' in LL(1) stands for scanning the input from left to right, the second 'L' for producing a leftmost derivation and the '1' for using one input symbol of lookahead at each step to make parsing action decisions.

For a given grammar an input is accepted or rejected that can be checked by using a LL (1) grammar, for this reason we need to find the value of First and Follow of that grammar.

5.2.1.2. Algorithm

First:

If X is a terminal then First(X) is just X!

If there is a Production $X \rightarrow \epsilon$ then add ϵ to first(X)

If there is a Production $X \rightarrow Y_1Y_2.. Y_k$ then add first ($Y_1Y_2..Y_k$) to first(X)

First ($Y_1Y_2..Y_k$) is either

First (Y_1) (if First (Y_1) doesn't contain ϵ)

OR (if First (Y_1) does contain ϵ) then First ($Y_1Y_2..Y_k$) is everything in First (Y_1) <except for ϵ > as well as everything in First ($Y_2..Y_k$)

If First (Y_1) First (Y_2)..First (Y_k) all contain ϵ then add ϵ to First ($Y_1Y_2..Y_k$) as well.

Follow:

First put \$ (the end of input marker) in Follow(S) (S is the start symbol)

If there is a production $A \rightarrow aBb$, (where a can be a whole string) then everything in FIRST (b) except for ϵ is placed in FOLLOW (B).

If there is a production $A \rightarrow aB$, then everything in FOLLOW (A) is in FOLLOW (B)

If there is a production $A \rightarrow aBb$, where FIRST (b) contains ϵ , then everything in FOLLOW (A) is in FOLLOW (B)

5.2.1.3. Program Code

```

packageFirstFollow;
public class FirstFollow{

    public static void main(String arg[]) {
        String left[] = {"E", "A", "T", "B", "F"};
        String right[] = {"TA", "+TA/e", "FB/e", "*FB/e", "(E)/i"};
        String right1[][] = new String[10][3];
        for (inti = 0; i<right.length; i++) {
            right1[i] = right[i].split("/");
        }
        for (inti = 0; i<right.length; i++) {
            for (int j = 0; j < right1[i].length; j++) {
                System.out.print("\t" + right1[i][j]);
            }
            System.out.print("\n ");
        }
        String first[][] = new String[right.length][2];

        for (inti = 0; i<right.length; i++) {
            int k = i;
            for (int j = 0; j < right1[k].length; j++) {
                if (right1[k][j].charAt(0) < 'A' || right1[k][j].charAt(0) > 'Z') {
                    first[i][j] = "" + right1[k][j].charAt(0);
                } else {
                    for (int h = 0; h < left.length; h++) {
                        if (left[h].charAt(0) == (right1[k][j].charAt(0))) {
                            k = h;
                            j = -1;
                            break;
                        }
                    }
                }
            }
        }

        System.out.print("\nFirst");
        for (inti = 0; i<first.length; i++) {
            System.out.print("\n");
            for (int j = 0; j < first[i].length; j++) {
                System.out.print(" " + first[i][j]);
            }
        }
    }
}

```

```

    }
    String follow[][] = new String[10][20];
    intfcount[] = new int[10];
    follow[0][0] = "$";
    fcount[0] = 1;
    for (inti = 0; i<left.length; i++) {
    if (i> 0) {
    fcount[i] = 0;
    }
    System.out.print("\n");

    for (int j = 0; j <right.length; j++) {
    for (int h = 0; h < right1[j].length; h++) {
    if (right1[j][h].contains(left[i])) {
    int B = right1[j][h].indexOf(left[i]);
        String a = right1[j][h].substring(0, B);
        String b = right1[j][h].substring(B + 1, right1[j][h].length());

    if (b.isEmpty()) {
    for (int k = 0; k <fcount[j] && j != i; k++) {
    follow[i][fcount[i]++] = follow[j][k];
        }
        } else {
    if ((int) b.charAt(0) >= 'A' && (int) b.charAt(0) <= 'Z') {
    for (int k = 0; k <left.length; k++) {
    if (left[k].equalsIgnoreCase(b)) {
    for (int m = 0; m < first[k].length; m++) {
    if (first[k][m].equalsIgnoreCase("e")) {
        } else {
    follow[i][fcount[i]++] = first[k][m];
        }
        }
    }
    break;
        }
        }
    }
    for (int k = 0; k <fcount[j] && j != i; k++) {
    follow[i][fcount[i]++] = follow[j][k];
        }
        } else {
    follow[i][fcount[i]++] = b;

```

```

    }
    }
    }
    }
    }
    System.out.print("\nFOLLOW");
    for (inti = 0; i<left.length; i++) {
    System.out.print("\n");
    for (int j = 0; j <fcount[i]; j++) {
    System.out.print(" " + follow[i][j]);
    }
    }
    }
}

```

5.2.1.4. Result and Discussion

TA
 +TA e
 FB e
 *FB e
 (E) i

First	FOLLOW
E-> (I	\$)
A-> + e	\$)
T-> (I	+ \$) + \$)
B-> * e	+ \$) + \$)
F-> (I	* + \$) + \$) * + \$) + \$)

5.2.2. Title of Problem

Construct a program to calculate cost of a given instruction.

5.1.2.1. Problem Description

In order to do faster calculation, optimization, appropriate using of register and memory we need to calculate cost of an instruction of a block. We can calculate cost in many ways. Now I calculate cost using one of them bellow.

5.2.2.2. Algorithm

Cost = 1 + Cost_of_operand- 1 + Cost_of_operand-2 + Cost_of_result.

	<u>Example</u>	<u>Cost</u>
Memory address	x	1
Register	r0	0
Literal	9	0
Indirect Register	[r1]	1
Double Indirect	[[r1+34]]	2

5.2.2.3. Program:

```
package costfunction;
import java.util.Scanner;
public class CostFunction {

    static int cost = 0;
    public static void main(String[] args)

        Scanner input = new Scanner(System.in);
        while (true) {
            System.out.println("Enter code: ");
            String code = input.next();
            if(code.equals("Exit")){
                System.out.println("Exiting from code");
                break;
            }
            cost = 1;
            while (true) {
```

```

        String op1 = input.next();
        String op2 = input.next();
        if (code.equals("MOV")) {
            calculate(op1);
            calculate(op2);
        }
        else {
            calculate(op1);
            calculate(op2);
            calculate(op2);
        }
        break;
    }

    System.out.println("Cost: " + cost);
    cost = 1;
}
}

public static void calculate(String op) {
    if (op.startsWith("r")) {
    } else if (op.startsWith "[")) {
        cost++;
    } else if (op.startsWith("[[")) {
        cost += 2;
    } else {
        try {
            int i = Integer.parseInt(op);
        } catch (Exception ex) {
            cost++;
        }
    }
}
}
}
}

```

5.2.2.4. Result and Discussion

Sub 97, r5

Cost = 1+0+0+0 = 1

Add [r1], [[r1+34]]

Cost = 1+2+1+2 = 6

5.2.3. TITLE OF PROBLEM: A program to remove left recursion from a given grammar.

5.2.3.1. PROBLEM DESCRIPTION: A grammar is left-recursive if we can find some non-terminal A which will eventually derive a sentential form with itself as the left-symbol. Immediate left recursion occurs in rules of the form

$$A \rightarrow A\alpha \mid \beta$$

Indirect left recursion in its simplest form could be defined as:

$$\begin{aligned} A &\rightarrow B\alpha \mid C \\ B &\rightarrow A\beta \mid D, \end{aligned}$$

possibly giving the derivation $A \Rightarrow B\alpha \Rightarrow A\beta\alpha \Rightarrow \dots$

More generally, for the nonterminals $A_0, A_1, \dots, A_n$, indirect left recursion can be defined as being of the form:

$$\begin{aligned} A_0 &\rightarrow A_1\alpha_1 \mid \dots \\ A_1 &\rightarrow A_2\alpha_2 \mid \dots \\ &\vdots \\ A_n &\rightarrow A_0\alpha_{n+1} \mid \dots \end{aligned}$$

where $\alpha_1, \alpha_2, \dots, \alpha_n$ are sequences of nonterminals and terminals.

REMOVING LEFT RECURSION:

The general algorithm to remove immediate left recursion follows. Several improvements to this method have been made, including the ones described in "Removing Left Recursion from Context-Free Grammars", written by Robert C. Moore.^[5] For each rule of the form

$$A \rightarrow A\alpha_1 \mid \dots \mid A\alpha_n \mid \beta_1 \mid \dots \mid \beta_m$$

where:

- A is a left-recursive nonterminal
- α is a sequence of nonterminals and terminals that is not null ($\alpha \neq \epsilon$)
- β is a sequence of nonterminals and terminals that does not start with A .

replace the A -production by the production:

$$A \rightarrow \beta_1 A' \mid \dots \mid \beta_m A'$$

And create a new nonterminal

$$A' \rightarrow \epsilon \mid \alpha_1 A' \mid \dots \mid \alpha_n A'$$

5.2.3.2. ALGORITHM:

Assign an ordering $A_1, \dots, A_n$ to the nonterminals of the grammar.

```
for  $i:=1$  to  $n$  do begin
  for  $j:=1$  to  $i-1$  do begin
    for each production of the form  $A_i \rightarrow A_j \alpha$  do begin
      remove  $A_i \rightarrow A_j \alpha$  from the grammar
    for each production of the form  $A_j \rightarrow \beta$  do begin
      add  $A_i \rightarrow \beta \alpha$  to the grammar
    end
  end
end
transform the  $A_i$ -productions to eliminate direct left recursion
end
```

5.2.3.3. PROGRAM CODE:

Left_recursion.java:

```
package recursion;
import java.util.Scanner;
public class Left_recursion {

    public static void main(String[] args) {
        int n;
        String p_p_c, first;
        Scanner input = new Scanner(System.in);
        System.out.println("Enter production number:");
        n = input.nextInt();
        String p[] = new String[100];
        System.out.println("Enter production :");
        for (int i = 0; i < n; i++) {
            input = new Scanner(System.in);
            try {
                p[i] = input.nextLine();
            } catch (Exception e) {
            }
        }
        method ob = new method();
    }
}
```

```

        System.out.println("Your Required production removing left recuritions is
given bellow:");
        ob.chSt(p, n);
    }
}

```

Method.java:

```

package recursion;
import java.text.DecimalFormat;
public class method {

    String left_char, right_first_char;
    int new_lenght = 0, n1;

    public void chSt(String s[], int n) {
        String s2[];
        s2 = new String[100];
        String num;
        new_lenght = n - 1;
        char f;
        for (int i = 0; i < n; i++) {
            String string = s[i];
            int l = s[i].indexOf(">");
            left_char = s[i].substring(0, l);
            right_first_char = s[i].substring(l + 1, l + 2);
            if (left_char.equals(right_first_char)) {
                new_lenght = new_lenght + 1;
                s2[i] = left_char + ">" + left_char + "";
                s2[new_lenght] = left_char + ">" + s[i].substring(l + 2, s[i].length()) +
left_char + "" + "/E";
            } else {
                s2[i] = s[i];
            }
        }
        display_strings(s2, new_lenght);
    }
    void display_strings(String p[], int n) {

        for (int i = 0; i <= n; i++) {

```

```

        System.out.println("\t\t" + p[i]);
    }
}
}

```

5.2.3.4. RESULT AND DISCUSSION:

Enter production number:

3

Enter production :

A>ds

G>GFDDJS

F>Fdsd

Your Required production removing left recurctions is given bellow:

A>ds

G>G'

F>F'

G'>FDDJSG'/E

F'>dsdF'/E

ACKNOWLEDGEMENT

This is to be described here that this work is done under the supervision of Md. Ashraf Uddin, Lecturer of Department of Computer Science and Engineering, Jagannath University.

I am greatly thankful to him for his assistance and aid that motivated me to do this work properly and completely. This document will be helpful to those scholars who are really interested in compiler design and implementation.